

CRAFTING INTERPRETERS - ANSWER SHEET

Tyler Bacong

Chapters 8 and 9 end-of-chapter challenges

GitHub repository: <https://github.com/TylerBac0ng/crafting-interpreters-ch8-9>

CHAPTER 8 - STATEMENTS AND STATE

Challenge 1 - REPL accepts statements and expressions

Question: The REPL no longer supports entering a single expression and automatically printing its result value. Add support to the REPL to let users type in both statements and expressions. If they enter a statement, execute it. If they enter an expression, evaluate it and display the result value.

- The REPL uses a new `Parser.parseRepl()` instead of `parse()`. It sets an `allowExpression` flag.
- In `expressionStatement()`, if that flag is set and the expression is followed directly by end of input, the missing semicolon is allowed and `foundExpression` is set.
- `parseRepl()` returns the bare `Expr` in that case, otherwise the usual `List<Stmt>`. The flag is cleared after the first statement, so only a single trailing expression counts as "an expression".
- `Lox.runRepl()` runs statements normally. For an expression it calls a new `Interpreter.interpret(Expr)` and prints the value it returns. Running a file still uses `parse()`, so scripts must use semicolons.

```
// Parser.java
Object parseRepl() {
    allowExpression = true;
    List<Stmt> statements = new ArrayList<>();
    while (!isAtEnd()) {
        statements.add(declaration());
        if (foundExpression) {
            Stmt last = statements.get(statements.size() - 1);
            return ((Stmt.Expression) last).expression;
        }
        allowExpression = false;
    }
    return statements;
}

private Stmt expressionStatement() {
    Expr expr = expression();
    if (allowExpression && isAtEnd()) {
        foundExpression = true;
    } else {
        consume(SEMICOLON, "Expect ';' after expression.");
    }
    return new Stmt.Expression(expr);
}

// Lox.java
private static void runRepl(String source) {
    ...
    Object syntax = parser.parseRepl();
    if (hadError) return;
    if (syntax instanceof List) {
        interpreter.interpret((List<Stmt>) syntax);
    } else if (syntax instanceof Expr) {
        String result = interpreter.interpret((Expr) syntax);
        if (result != null) System.out.println(result);
    }
}
```

```
}
```

Example session (from tests/repl.input):

```
> 1 + 2
3
> var a = 10;
> a * 2
20
> print "stmt";
stmt
> a = a + 1
11
```

Challenge 2 - Accessing an uninitialized variable is a runtime error

Question: Instead of implicitly initializing variables to nil, make it a runtime error to access a variable that has not been initialized or assigned to.

- `var a;` can no longer store nil, because then `var a;` and `var a = nil;` would look the same. Only the second one should be allowed.
- The interpreter has a private sentinel object, `uninitialized`, that no Lox program can produce. `visitVarStmt` stores it when there is no initializer.
- `visitVariableExpr` checks for the sentinel and throws a `RuntimeError`. Assignment just replaces the value, so assigning a variable before reading it still works.

```
private static final Object uninitialized = new Object();

public Void visitVarStmt(Stmt.Var stmt) {
    Object value = uninitialized;
    if (stmt.initializer != null) value = evaluate(stmt.initializer);
    environment.define(stmt.name.lexeme, value);
    return null;
}

public Object visitVariableExpr(Expr.Variable expr) {
    Object value = environment.get(expr.name);
    if (value == uninitialized) {
        throw new RuntimeError(expr.name,
            "Variable '" + expr.name.lexeme + "' must be initialized before use.");
    }
    return value;
}
```

Test (tests/ch8_uninitialized.lox):

```
var a = nil; print a;      // nil      (explicit nil is fine)
var b; b = "assigned"; print b;    // assigned
var c; print c;           // Variable 'c' must be initialized before use. [line 7]
```

Challenge 3 - `var a = a + 2;` inside a nested block

Question: What does the following program do? What did you expect it to do? Is it what you think it should do? What does analogous code in other languages you are familiar with do? What do you think users will expect this to do?

```
var a = 1;
{
    var a = a + 2;
    print a;
}
```

- **What it does:** it prints 3. `visitVarStmt` evaluates the initializer *before* it calls `define()`. At that point the inner `a` does not exist yet, so the lookup finds the outer `a = 1`. Then a new inner `a = 3` is defined, and the outer `a` stays 1. (Confirmed by `tests/ch8_shadow.lox`, which prints 3 then 1.)
- **What I expected:** 3. The initializer reads like it should use the `a` that already exists, and in `jlox` that is what happens.
- **Is it what it should do?** No. The line uses one name for two different variables. It almost always means a mistake, and the result depends on the exact moment the new variable comes into scope. It would be better to reject it as an error ("can't read local variable in its own initializer"). The book adds exactly this check in the resolver in Chapter 11.
- **Other languages:**
 - * C: the inner `a` is in scope from its own declarator, so `a + 2` reads the new, uninitialized `a`. That is undefined behavior (garbage output). Compilers can warn about it.
 - * Java: a compile error. A local variable may not shadow another local in an enclosing block ("variable `a` is already defined").
 - * JavaScript: with `let`, a `ReferenceError` ("Cannot access 'a' before initialization"), because the inner `a` is in its temporal dead zone. With `var` there is no block scope, so it is the same variable: it prints 3 and changes the outer `a` too.
 - * Rust / OCaml / Scheme `let`: the new binding is not in scope inside its own initializer, so it reads the outer `a` and prints 3, the same as `jlox`. (Scheme's `let rec` would be an error instead.)
 - * Python: blocks do not create scopes, so this is just `a = a + 2` on the same variable.
- **What users will expect:** most will expect 3, meaning "make a new `a` from the old `a`". Users coming from C or Java may expect an error or a warning instead. Because users disagree and C gives garbage, a clear compile-time error is the safest choice.

CHAPTER 9 - CONTROL FLOW

Challenge 1 - Branching with first-class functions and dynamic dispatch

Question: When `Lox` supports first-class functions and dynamic dispatch, we technically won't need branching statements built into the language. Show how conditional execution can be implemented in terms of those. Name a language that uses this technique for its control flow.

- Make `true` and `false` two different classes. Each has the same method, which takes the "then" code and the "else" code as functions. `True` calls the first one and `False` calls the second.
- Choosing the branch is now done by method lookup, since the method that runs depends on the receiver's class. The language needs no `if`.
- The branches must be passed as functions (closures), not values. Otherwise both branches would run before the call. Wrapping them in functions delays them so that only the chosen one runs.

```
class True {
  ifThenElse(thenBranch, elseBranch) { return thenBranch(); }
}
class False {
  ifThenElse(thenBranch, elseBranch) { return elseBranch(); }
}
```

```
var yes = True();
var no = False();
```

```
fun sayYes() { print "yes"; }
fun sayNo() { print "no"; }
```

```
yes.ifThenElse(sayYes, sayNo); // prints "yes"
no.ifThenElse(sayYes, sayNo);  // prints "no"
```

- This uses functions and classes from Chapters 10-12, so it cannot run on the Chapter 9 interpreter. It is saved as `examples/ch9_branching.lox` in the repository, and running it with the book's finished jlox prints yes then no.
- **Language:** Smalltalk. Boolean has subclasses True and False, and conditionals are ordinary messages that take blocks: `(x > 3) ifTrue: [ ... ] ifFalse: [ ... ]`. Self works the same way. Church booleans in the lambda calculus are the same idea without classes.

Challenge 2 - Looping with recursion

Question: Likewise, looping can be implemented using those same tools, provided our interpreter supports an important optimization. What is it, and why is it necessary? Name a language that uses this technique for iteration.

- A loop can be written as a function that runs the body once and then calls itself for the next iteration. The version below uses no while, for, or if. It uses the True/False classes from Challenge 1 to decide whether to go around again:

```
// True and False classes from Challenge 1 go here.
```

```
fun whileLoop(condition, body) {
  fun next() {
    body();
    return whileLoop(condition, body); // tail call
  }
  fun stop() { return nil; }
  return condition().ifThenElse(next, stop);
}

var i = 0;
var t = True();
var f = False();
fun check() { return (i < 5 and t) or f; } // Lox boolean -> True/False object
fun step() { print i; i = i + 1; }

whileLoop(check, step); // prints 0 1 2 3 4
```

- This is saved as `examples/ch9_looping.lox`. Running it with the book's finished jlox prints 0 through 4.
- **The optimization:** tail call optimization (TCO), also called tail call elimination. A call is a tail call when it is the last thing a function does before returning, so the caller's stack frame is no longer needed. With TCO the callee reuses that frame instead of pushing a new one.
- **Why it is necessary:** without TCO, each iteration adds a stack frame that is only freed when the whole loop ends. Memory use grows with the number of iterations, and a long loop (or an infinite one, like a server's main loop) will overflow the stack. In jlox a Lox call is a Java call, and jlox has no TCO. When the example above is changed to count to 100,000, it crashes with `java.lang.StackOverflowError`, but a while loop of that length runs fine. TCO makes recursive loops use constant stack space, just like while. In a tree-walker it could be done with a trampoline: a tail call returns a "call this next" object, and the caller runs it in a loop.
- **Language:** Scheme. The language standard requires proper tail calls, and iteration is written as tail recursion (named `let`, and `do`, which is defined in terms of it). Many functional languages such as Haskell, Erlang, and Elixir also have no built-in loop statements and depend on tail calls.

Challenge 3 - break statement

Question: Add support for break statements. The syntax is a break keyword followed by a semicolon. It should be a syntax error to have a break statement appear outside of any enclosing loop. At runtime, a break statement causes execution to jump to the end of the nearest enclosing loop and proceeds from there. Note that the break may be nested inside other blocks and if statements that also need to be exited.

- Grammar: `statement -> ... | breakStmt ; breakStmt -> "break" ";" ;`
- Scanner/AST: a new BREAK keyword token and a `Stmt.Break(Token keyword)` node, added to `GenerateAst`. BREAK is also a `synchronize()` point.
- **Syntax error outside a loop:** the parser keeps a `loopDepth` counter. It goes up while parsing a while or for body and back down afterwards, in a finally block so parse errors do not leave it wrong. `breakStatement()` reports an error if the counter is 0. The error is reported without being thrown, so the parser does not need to resynchronize.

```
private Stmt breakStatement() {
    Token keyword = previous();
    if (loopDepth == 0) {
        error(keyword, "Must be inside a loop to use 'break'.");
    }
    consume(SEMICOLON, "Expect ';' after 'break'.");
    return new Stmt.Break(keyword);
}
```

```
private Stmt whileStatement() {
    ...
    try {
        loopDepth++;
        Stmt body = statement();
        return new Stmt.While(condition, body);
    } finally {
        loopDepth--;
    }
}
// forStatement() wraps its body the same way.
```

- **Runtime:** break must leave any number of blocks and if statements, which are nested Java calls in the interpreter. So it throws a `BreakException`, and `visitWhileStmt` catches it. Each block on the way out restores its environment in `executeBlock`'s finally, so scoping stays correct. The exception turns off stack traces because it is only used for control flow.
- for loops are turned into while loops, so they get break for free. Skipping the increment on a break is correct, since the loop is ending anyway. The innermost loop catches the exception first, so only the nearest enclosing loop exits.

```
private static class BreakException extends RuntimeException {
    BreakException() { super(null, null, false, false); } // no stack trace
}

public void visitBreakStmt(Stmt.Break stmt) {
    throw new BreakException();
}

public void visitWhileStmt(Stmt.While stmt) {
    try {
        while (isTruthy(evaluate(stmt.condition))) {
            execute(stmt.body);
        }
    } catch (BreakException ex) {
        // Exit the loop.
    }
}
```

```
    return null;
}
```

Tests (tests/ch9_break.lox and tests/ch9_break_outside.lox):

```
var i = 0;
while (true) { if (i == 3) break; print i; i = i + 1; }    // 0 1 2
for (var x = 0; x < 3; x = x + 1)
    for (var y = 0; y < 10; y = y + 1) {
        if (y == 2) break;                                // exits only the inner loop
        print x * 10 + y;                                  // 0 1 10 11 20 21
    }
```

```
break;                // [line 2] Error at 'break': Must be inside a loop to use 'break'.
if (true) break;      // [line 3] Error at 'break': Must be inside a loop to use 'break'.
```

All five tests pass with ./run_tests.sh.